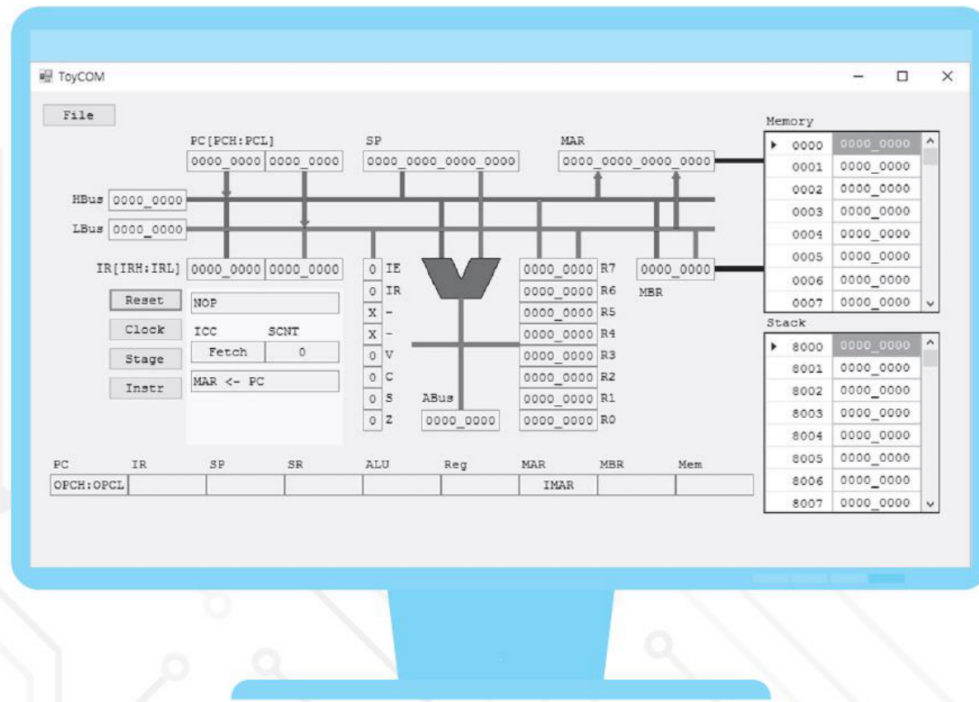


한눈에 보이는

개정 3 판

컴퓨터 구조



06. 명령어 집합

6.1 명령어 특성

6.2 주소의 수

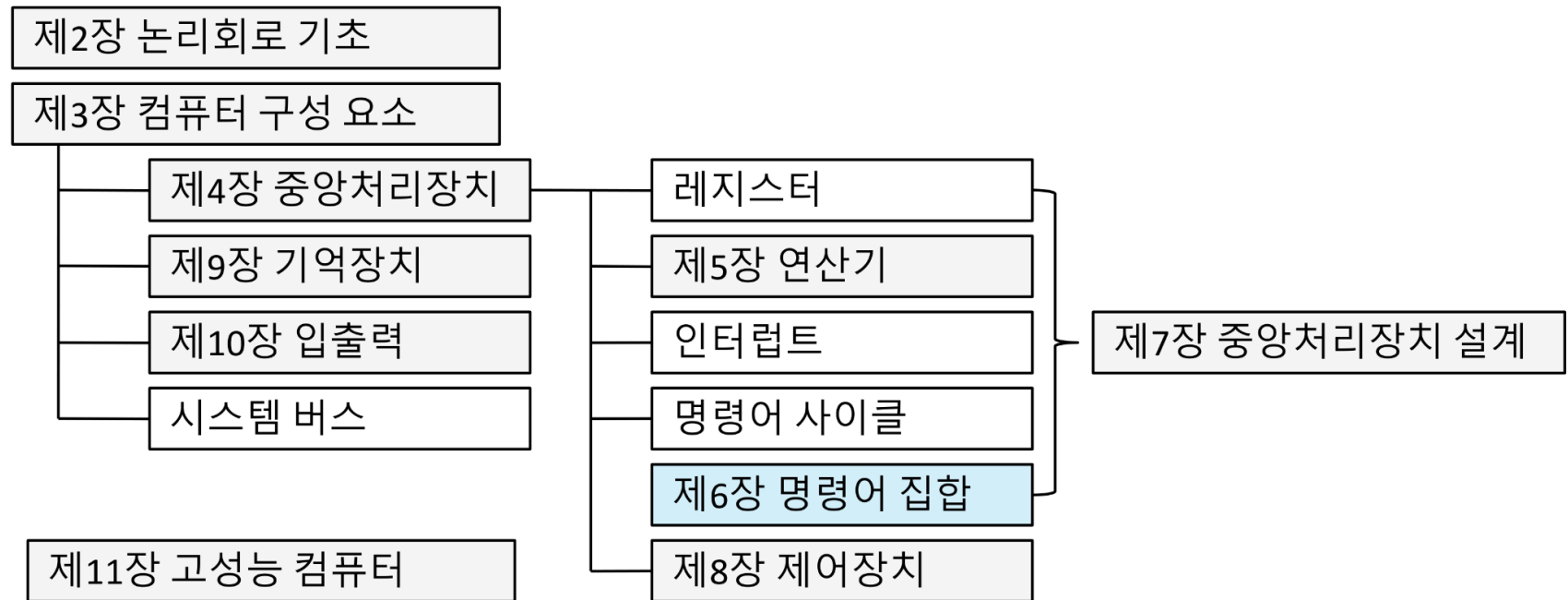
6.3 주소지정방식

6.4 오퍼랜드 저장

6.5 명령어 종류

6.6 요약

제6장 명령어 집합



학습 목표

- 명령어를 표현하는 방법을 이해한다.
- 오퍼랜드 필드가 지정하는 유효 데이터를 구할 수 있다.
- 기억장치에 저장되어 있는 단어의 값을 해석할 수 있다.

6.1 명령어 특성

동작 코드와 오퍼랜드 소개

한정된 비트에 명령어 구성 요소를 배치하는 방법

내용

- 6.1.1 명령어 구성 요소
- 6.1.2 명령어 종류
- 6.1.3 명령어 형식

6.1.1 명령어 구성 요소

명령어 구성 요소: 동작코드 + 오퍼랜드

동작코드(opcode, operation code)

- 중앙처리장치가 실행할 동작을 2진수로 표현한 코드

☆ 니모닉 코드(mnemonic code): 의미를 나타내는 기호 코드

- 예) ADD, SUB, MUL, DIV, BR, PUSH, POP

오퍼랜드(operand) ↳ branch (≈ if ㄱ)

- 동작의 대상: 소스 오퍼랜드, 목적지 오퍼랜드, 분기목적지 주소
- 실제: 레지스터 번호, 기억장치 주소, 입출력 포트

ex) mnemonic code

Pentium: MOV, EDX, ADD...

ARM: SUB, BNE..

6.1.2 명령어 종류

명령어 종류

- 데이터 전달 명령어: $A \leftarrow B$
- 데이터 처리 명령어: $A \leftarrow B \text{ op } C$
- ◈ 프로그램 제어 명령어: $PC \leftarrow \text{목적지 주소}$

프로그램 실행 순서를 변경 (= PC를 변경)

6.1.3 명령어 형식

명령어 형식

- 명령어 비트 영역에 명령어 구성 요소를 배치한 형태
- 동작 코드 필드, 오퍼랜드 필드

프로세서의 명령어 형식: 고정 길이/가변 길이 명령어 형식

고정 길이 (fixed-length) 명령어 형식 *fixed length instruction format*

- 명령어의 길이가 모두 같다.
- 해석이 쉽고, 제어장치가 간단하다.
- RISC (Reduced Instruction Set Computer)

[Risk]

가변 길이 (variable-length) 명령어 형식 *variable length instruction format*

- 명령어 종류별로 길이가 다르다.
- 해석이 어렵고, 제어장치가 복잡하다.
- CISC (Complex Instruction Set Computer)

PowerPC 명령어 형식

옛날 CPU
RISC

대부분의 모바일은 RISC

6비트	5비트	5비트	16비트	
branch	long immediate			A/L
cond. br.	options	CR Bit	displacement	A/L

(a) 분기 명령어

cond. reg.	dest. bit	src. bit	src. bit	add, or, xor, etc.	X
------------	-----------	----------	----------	--------------------	---

(b) 조건부 레지스터 논리 명령어

ld/st indirect	dest. reg.	base reg.	displacement		
ld/st indirect	dest. reg.	base reg.	index reg.	size, sign, update	X

(c) 적재/저장 명령어

arithmetic	dest. reg.	src. reg.	src. reg.	sub-opcode
arithmetic	dest. reg.	src. reg.	immediate value	

(d) 산술 연산 명령어

float sgl/dbl	dest. reg.	src. reg.	src. reg.	src. reg.	sub-op	R
---------------	------------	-----------	-----------	-----------	--------	---

(e) 실수 연산 명령어

32 비트로 고정

그림 6-1 PowerPC 프로세서의 명령어 형식

Pentium 명령어 형식

CISC

[Sisk]

대부분의 PC는 CISC

계열 CPU들은 원만하면 CISC
(하위호환을 보장해야하므로)

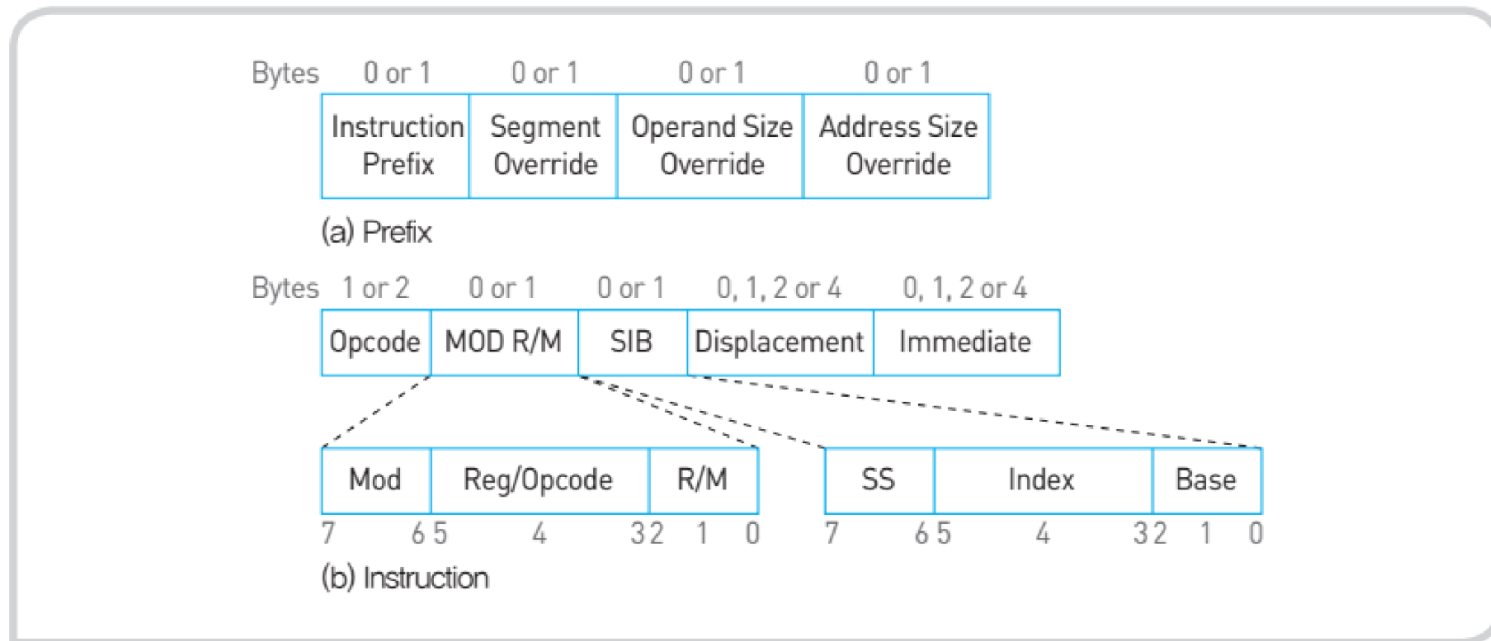


그림 6-2 Pentium 프로세서의 명령어 형식

6.2 주소의 수

명령어 종류 별 오퍼랜드

- 데이터 전달 명령어: 목적지, 소스
- 데이터 처리 명령어: 목적지, 소스1, 소스2
- 프로그램 제어 명령어: 분기 목적지

학습목표

- 명령어에 표현된 오퍼랜드를 해석할 수 있다.

내용

(오퍼랜드 개수로 생각)

- 6.2.1 3-주소 명령어 형식 (3-address instruction format) 오퍼랜드 3개
- 6.2.2 2-주소 명령어 형식 (2-address instruction format)
- 6.2.3 1-주소 명령어 형식 (1-address instruction format)
- 6.2.4 0-주소 명령어 형식 (0-address instruction format) : 스택 연산

RISC: 저가, 산업용

CISC: 고가, 주로 PC용

6.2.1 3-주소 명령어 형식

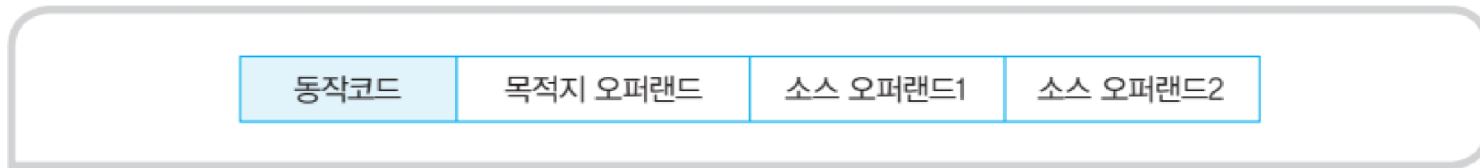


그림 6-3 3-주소 명령어 형식

목적지 $\leftarrow$ 소스1 op 소스2

$Y = A \times B + C \times D$

- MUL R1, A, B // $R1 \leftarrow A \times B$
- MUL R2, C, D // $R2 \leftarrow C \times D$
- ADD Y, R1, R2 // $Y \leftarrow R1 + R2$

특징

- 오퍼랜드를 3개 모두 표현한다.
- 명령어 길이가 길어진다.
- 프로그램을 구현하는 명령어 수가 적다.
- RISC 프로세서에서 주로 사용한다.

6.2.2 2-주소 명령어 형식

동작코드	목적지/소스 오퍼랜드	소스 오퍼랜드

그림 6-4 2-주소 명령어 형식

목적지 $\leftarrow$ 목적지 op 소스

$Y = A \times B + C \times D$

- LD R1, A // $R1 \leftarrow A$
- MUL R1, B // $R1 \leftarrow R1 \times B$
- LD R2, C // $R2 \leftarrow C$
- MUL R2, D // $R2 \leftarrow R2 \times D$
- ADD R1, R2 // $R1 \leftarrow R1 + R2$
- ST Y, R1 // $Y \leftarrow R1$

특징

- 소스 오퍼랜드 하나를 생략한다.
- 명령어 길이가 짧아진다.
- 프로그램을 구현하는 명령어 수가 증가한다.
- CISC 프로세서에서 주로 사용한다.

처리량에 더 많은 한데 기본적인 클럭 자체가
CISC가 훨씬 빨라서 전체적인 속도는 더 빠름

6.2.3 1-주소 명령어 형식 이거는 안쓰

(초기 컴퓨터에서 쓸)



그림 6-5 1-주소 명령어 형식

$AC \leftarrow AC \text{ op } \text{오퍼랜드}$

$Y = A \times B + C \times D$

- LD A // $AC \leftarrow A$
- MUL B // $AC \leftarrow AC \times B$
- ST T // $T \leftarrow AC$
- LD C // $AC \leftarrow C$
- MUL D // $AC \leftarrow AC \times D$
- ADD T // $AC \leftarrow AC + T$
- ST Y // $Y \leftarrow AC$

특징

- 누산기(AC)를 사용한다.
- 명령어 표현에서 누산기를 생략한다.
- 명령어의 길이가 짧다.
- 프로그램을 구현하는 명령어의 수가 가장 많다.
- 초기 컴퓨터에서 사용하였으나, 지금은 찾아보기 힘들다.

X 6.2.4 0-주소 명령어 형식 애는 진짜 안씀

특수 계산기에서 사용하는 명령어 형식

스택 연산

- 계산식 표현
- 후위 표기식: $Y = A \times B + C \times D \Rightarrow (Y = AB \times CD \times +)$

계산 과정

- 변수: PUSH Data // 데이터를 푸시
- 연산자: POP R1 // 두 개의 데이터를 팝하고
 POP R2
 R3 $\leftarrow$ R1 op R2 // 계산 후
 PUSH R3 // 결과를 푸시
- 계산 끝: POP Y // 계산 결과를 변수에 저장

스택 연산 한번 보셈

후위 표기식: $(Y = AB \times CD \times +)$

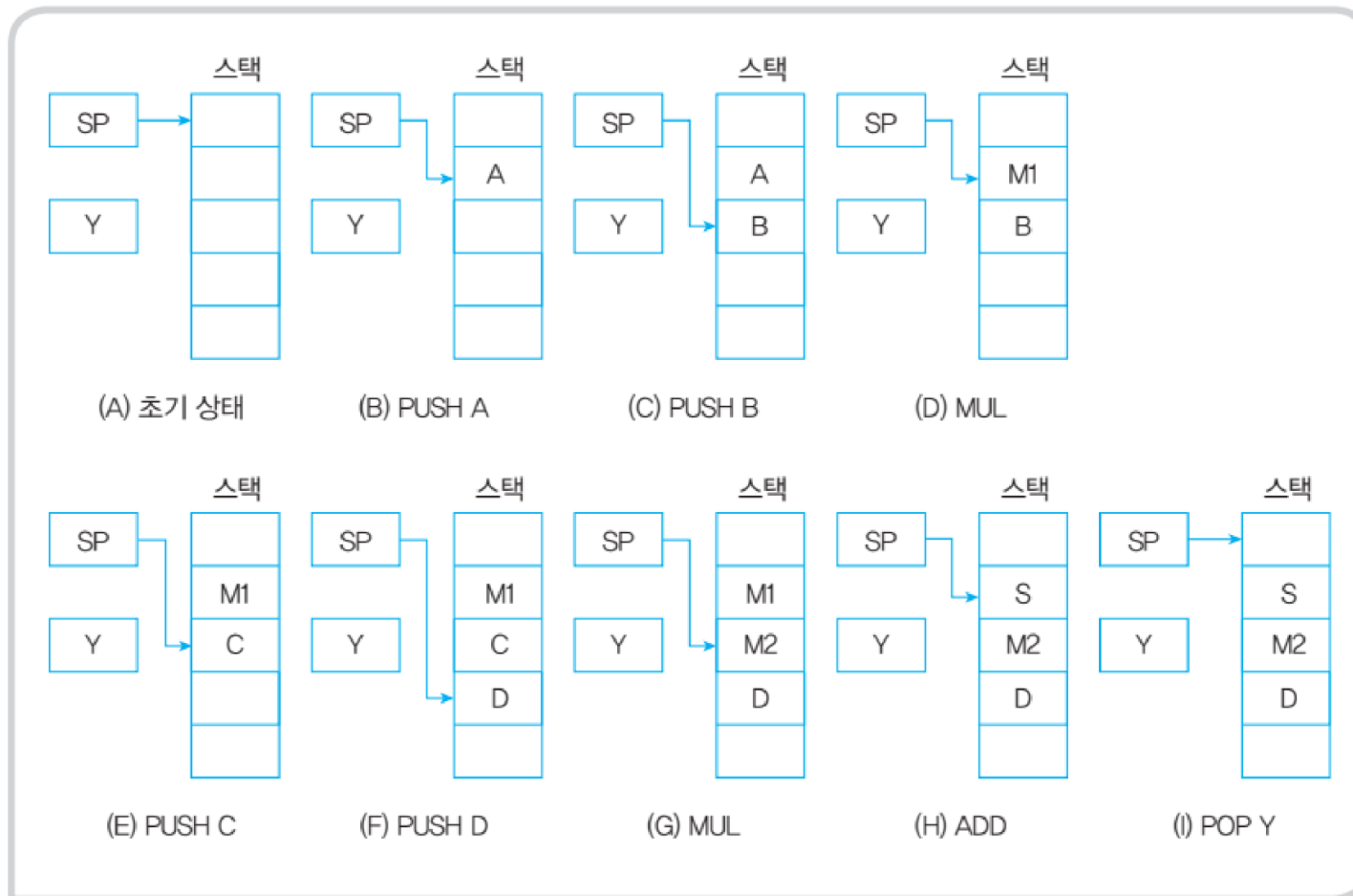


그림 6-6 스택 연산

6.3 주소지정방식

용어

- 오퍼랜드 필드: 명령어에 표현되어 있는 연산의 대상
- 유효 데이터(effective data): 실제로 처리되는 데이터
- 유효 주소(effective address): 유효 데이터가 저장되어 있는 기억장치 주소

학습 목표

- 명령어의 오퍼랜드 필드에서 유효 데이터를 찾을 수 있다.

내용

- 6.3.1 즉시 주소지정방식(immediate addressing mode)
- 6.3.2 직접 주소지정방식(direct addressing mode)
- 6.3.3 간접 주소지정방식(indirect addressing mode)
- 6.3.4 레지스터 주소지정방식(register addressing mode)
- 6.3.5 레지스터 간접 주소지정방식(register indirect addressing mode)
- 6.3.6 변위 주소지정방식(displacement addressing mode)

주소지정방식

주소지정방식

- 오퍼랜드 필드마다 별도 적용

표기법

- A: 명령어에 표현된 주기억장치 주소
- Mem(A): 주기억장치 A 번지에 저장된 값 $\approx A^*$
- R: 레지스터 번호
- (R): 레지스터 R에 저장되어 있는 값
- EA: 유효 주소(effective address).
유효 데이터가 저장되어 있는 기억장치 주소.

6.3.1 즉시 주소지정방식

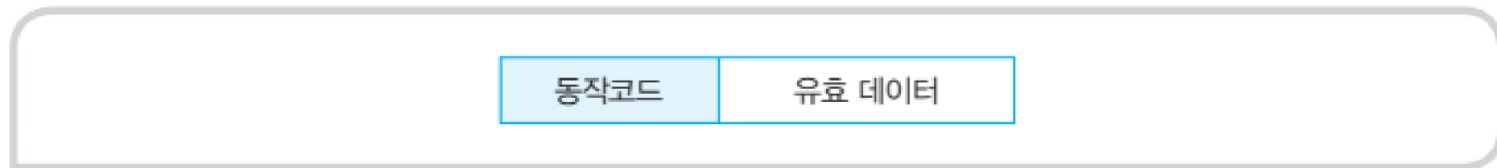


그림 6-7 즉시 주소지정방식

즉치 데이터(immediate data)

- 명령어에 유효 데이터 포함
- 명령어를 실행할 때 기억장치를 액세스할 필요가 없다.
- 유효 데이터의 표현 범위가 오퍼랜드 필드 크기에 제한 받는다.

명령어 예

- 1-주소지정방식
- 2-주소지정방식

ADDI #20 // AC $\leftarrow$ AC + 20
LDI R0, #12 // R0 $\leftarrow$ 12

주소변수가 아닌
데이터 자체가 20임을 의미

x 즉치 주소지정방식

[예제 6-1]

명령어 길이 16비트 = { 동작코드:5비트, 오퍼랜드:11비트 }

- 1) 즉치 데이터로 부호 없는 수를 표현할 수 있는 수의 범위는?
- 2) ADDI #12 명령어의 비트 패턴은? (동작코드는 10010)
- 3) 누산기의 값이 5일 때, ADDI #12 명령어를 실행 후 값은?

6.3.2 직접 주소지정방식

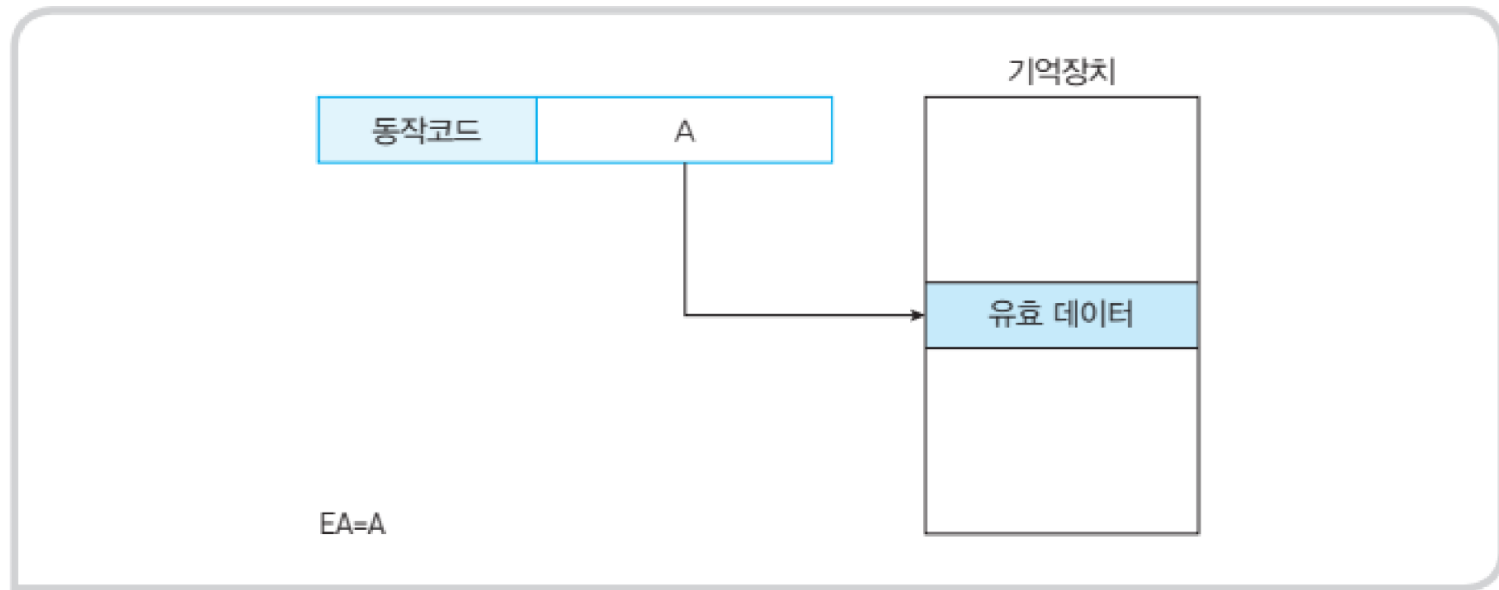


그림 6-8 직접 주소지정방식

직접 주소지정방식 (direct addressing mode)

- 명령어에 기억장치 주소(A)가 표현된다. A는 부호 없는 수이다.
- 유효 주소 = A

명령어 예

- 1-주소지정방식
- 2-주소지정방식

여기서는 20 이 주소이름을 의미

ADD #20	// AC ← AC + Mem(20)
LD R0, #1200	// R0 ← Mem(1200)

x 직접 주소지정방식

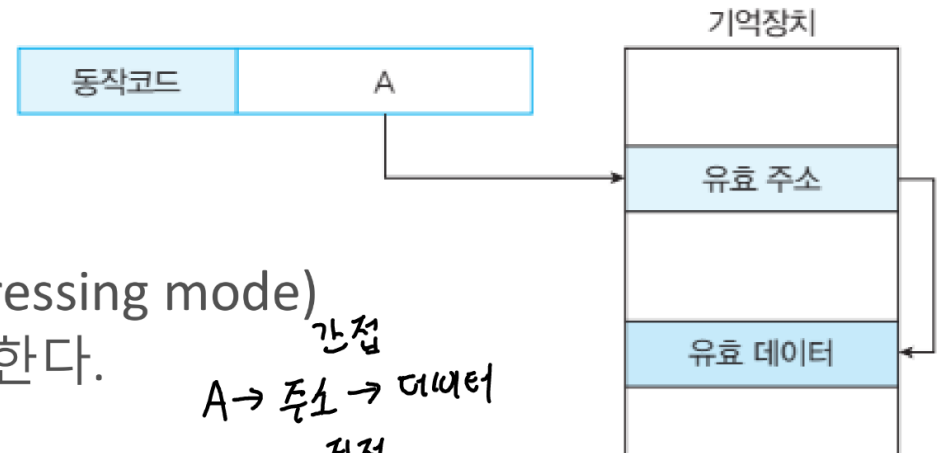
[예제 6-2]

- 1-주소 명령어 형식
 - 명령어 길이 16비트 = {동작코드:5비트, 오퍼랜드:11비트}
- 1) 직접 주소지정방식으로 지정할 수 있는 기억장치 용량은?
 - 2) 명령어 LD #123의 비트 패턴은? (123은 10진수)
 - 3) LD #123을 실행한 후 AC의 값은?

주소	데이터
...	...
125	120
124	122
123	124
122	126
121	128
120	130

그림 6-9 예제 6-2의 기억장치 상태

6.3.3 간접 주소지정방식 ☆ 아를 많이 쓴다



간접 주소지정방식(indirect addressing mode)

- 명령어에 기억장치 주소를 표현한다.
- A는 부호 없는 수이다.
- A의 길이는 기억장치 주소의 길이와 같다.
- 유효 주소 = mem(A)
- 명령어 실행 단계에서 기억장치를 2번 액세스한다.
 - 유효 주소 인출/유효 데이터 인출 (직접은 1번)

간접
A → 주소 → 데이터
직접
A → 데이터

명령어 예

- 1-주소지정방식 `ADDID #20` // $AC \leftarrow AC + \text{Mem}(\text{Mem}(20))$
- 2-주소지정방식 `LDID R0, #1200` // $R0 \leftarrow \text{Mem}(\text{Mem}(1200))$

↳ 1200번지에는 어떠한 주소가 있고,
그 주소의 값을 R0에 넣으라

× 간접 주소지정방식

[예제 6-3]

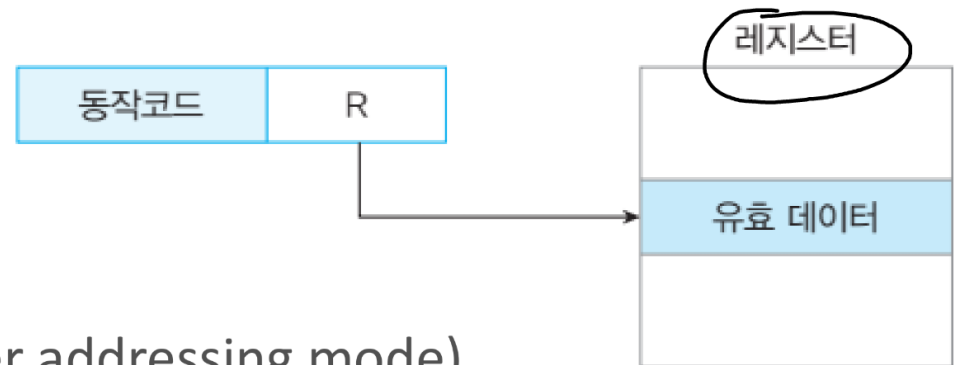
- 1-주소 명령어 형식
- 명령어 길이 16비트 = {동작코드:5비트, 오퍼랜드:11비트}

- 1) 명령어 LDID #121의 비트 패턴은? (121은 10진수)
- 2) LDID #121을 실행한 후 AC의 값은?

주소	데이터
...	...
125	120
124	122
123	124
122	126
121	125
120	130

그림 6-9 예제 6-2의 기억장치 상태

10.3.4 레지스터 주소지정방식



레지스터 주소지정방식(register addressing mode)

- 명령어는 레지스터 번호 표현
- 유효 주소 개념이 없음. (‘.’ 기억장치 사용X)
- R 필드이 길이는 주소(A) 필드 길이보다 많이 짧다.

레지스터는 기억장치보다 많이 작음

데이터가 메모리가 아닌
레지스터에 저장되어있음

명령어 예

- 2-주소지정방식 LD R0, #1200 // R0 ← Mem(1200)
- 2-주소지정방식 ADD R0, R1 // R0 ← R0 + R1
- 3-주소지정방식 ADD R0, R1, R2 // R0 ← R1 + R2

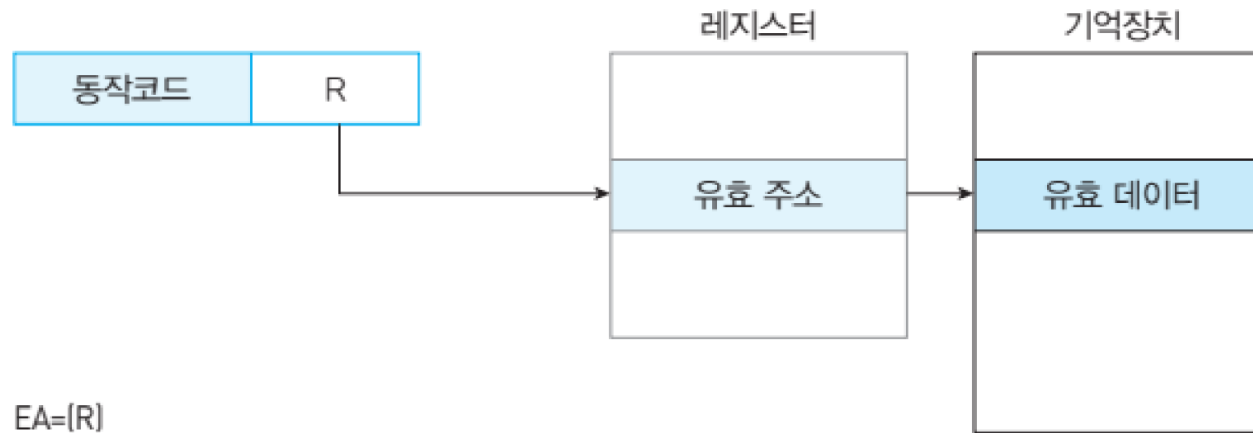
레지스터 주소지정방식

[예제 6-4]

- 프로세서 레지스터: R0 ~ R15 (16개)
- 3-주소 형식의 연산 명령어
- 명령이 길이 16비트
= {동작 코드:4비트, 목적지:4비트, 소스1:4비트, 소스2:4비트}

- 1) ADDR R0, R1, R2 명령어의 비트 패턴은? (ADDR=0010, $R_d \leftarrow R_{s1} + R_{s2}$)
- 2) R0=2, R1=3, R2=2일 때, 명령어 실행 후 변경되는 레지스터의 값은?

6.3.5 레지스터 간접 주소지정방식 ~~다들~~ 많이 쓴다



레지스터 간접 주소지정방식(register indirect addressing mode)

- 명령어는 레지스터 번호를 표현 ~~레지스터~~ → 지정하는데 필요한 용량 작음 → 반도제 크기 ↓ → 돈
- 유효 주소 = (R)
- 유효 데이터 = mem((R))
- 실행단계에서 유효 데이터를 인출하기 위하여 기억장치를 한 번 액세스
- 명령어의 길이가 짧으면서, 기억장치를 액세스할 수 있다.

X 레지스터 간접 주소지정방식

[예제 6-5] 적재 명령어와 저장 명령어의 동작이 다음과 같다.

- 레지스터 간접 적재 명령어: LDR Rd, Mem(Rs+1:Rs) // $Rd \leftarrow \text{Mem}(Rs+1:Rs)$
 - 레지스터 간접 저장 명령어: STR Mem(Rd+1:Rd), Rs // $\text{Mem}(Rd+1:Rd) \leftarrow Rs$
- 적재 명령어는 Rs만 표현하고 Rs+1을 표현하지 않고, 저장 명령어는 Rd만 표현하고 Rd+1을 표현하지 않는다.

예를 들어, 적재 명령어의 Rs=6이면, 기억장치 주소는 (R7:R6)이다.

- 1) LDR R0, Mem(R7:R6) 명령어를 실행한 후 변하는 레지스터의 값을 구하라.
- 2) STR Mem(R7:R6), R1 명령어를 실행한 후 변하는 기억장치의 값을 구하라.

5비트	3비트	3비트	3비트	2비트	주소	데이터	주소	데이터
LDR: 00110	Rd	Rs	xxx	00	R7	01	...	...
STR: 00110	Rd	Rs	xxx	01	R6	23	125	78
					...	...	124	AB
					R1	3A	123	0D
					R0	00	122	1A

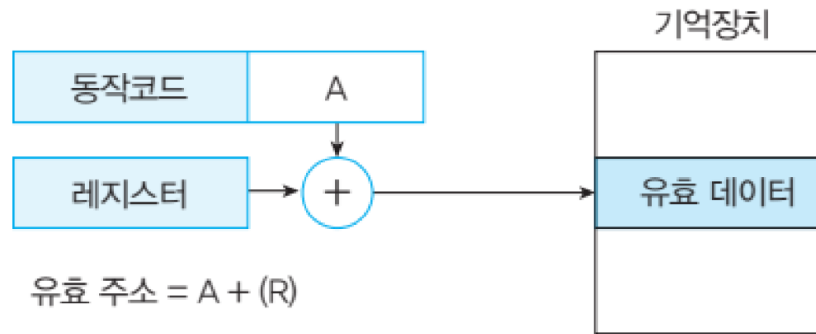
(a) 명령어 형식

(b) 레지스터 상태

(c) 기억장치 상태

그림 6-13 예제 6-5의 명령어 형식과 상태

6.3.6 변위 주소지정방식 ^{Displacement} ~~6~~ _{매도 많음}



유효 주소 = $(R) + A$ // 부호 확장 덧셈 $\underbrace{\text{주소} + \text{주소}}_{\text{새로운 주소}} \rightarrow \text{데이터}$
_{레지스터의 내용물 + A}

부호 확장 덧셈 예

- 기억장치 주소: 16 비트 (기억장치 공간 64 Kbytes)
- 레지스터 길이: 16 비트
- 명령어 주소 필드 A: 12 비트
- 유효 주소 = $(R)_{(16\text{비트})} + A_{(12\text{비트})}$

장점: A의 길이 줄이기 가능

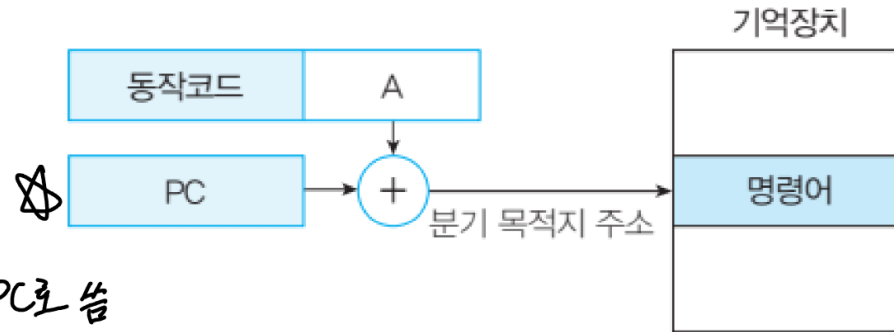
레지스터 주소는 A 해서 나온 주소로 확장함

종류

- 레지스터를 어떤걸로 쓰는지?
- 상대 주소지정방식(relative addressing mode)
 - 베이스 레지스터 주소지정방식(base-register addressing mode)
 - 인덱싱(indexing)

if, for, while 등에 쓰임

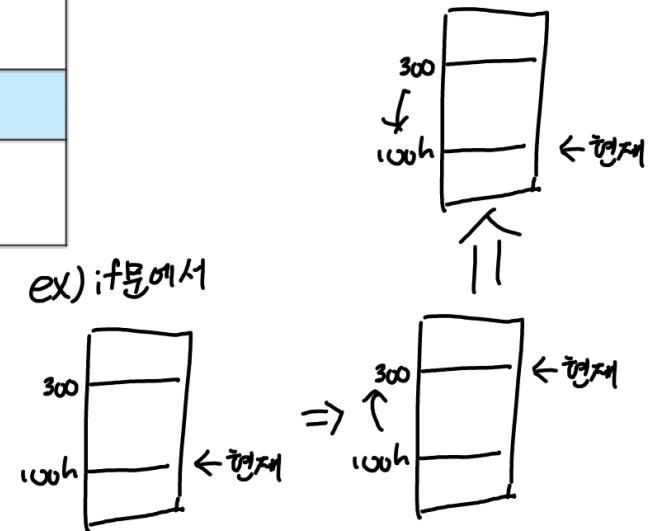
상대 주소지정방식 ~~제어~~ ~~필요~~



레지스터를 PC로 씀

상대 주소지정방식 (relative addressing mode) ex) if문에서

- 분기 목적지 = $PC + A$ (정수, 부호 확장)
- (조건) 분기 명령어, 서브루틴 호출 명령어

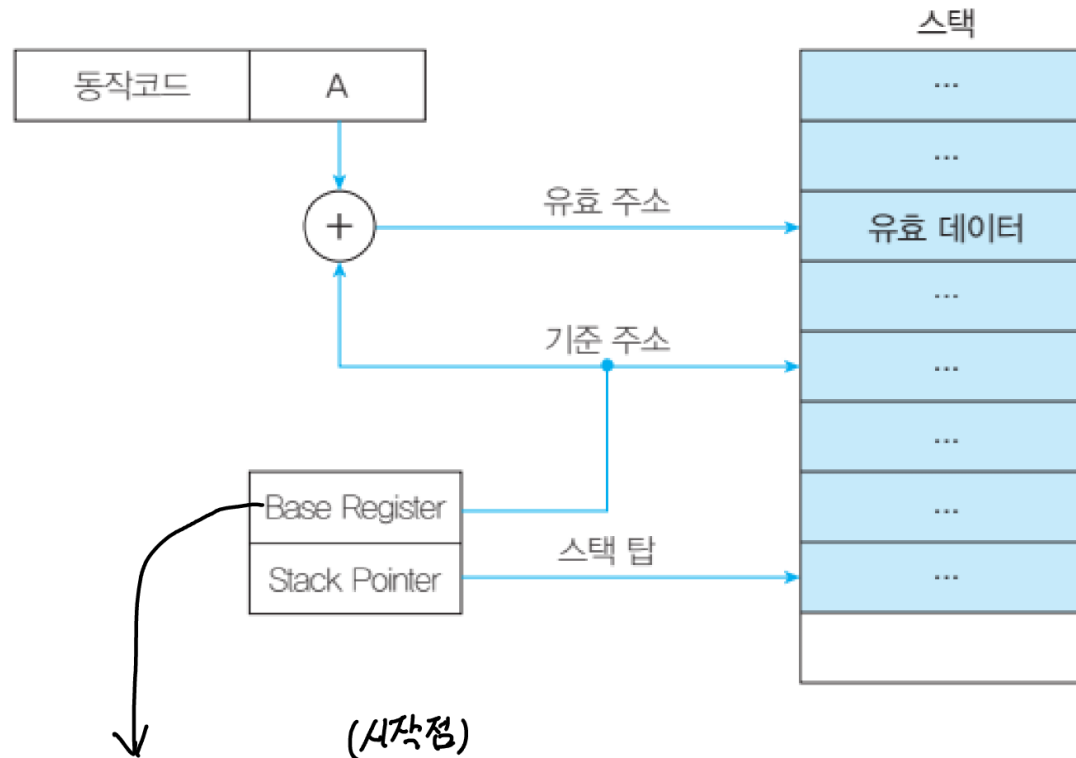


[예제 6-6]

- 기억장치 용량: 64 KB, 바이트 단위 주소, 명령어 크기 16비트 (2바이트)
- BR 명령어의 동작 코드는 11011, 나머지는 11비트는 상대 주소
- 현재 프로세서는 204_{10} 번지에 있는 무조건 분기(BR) 명령어 실행 중

- 1) 현재 PC의 값은? 206_{10}
- 2) 명령어 코드 11011_000_0100_0000 실행 후 PC는? $PC + 코드 \rightarrow 206 + 64 = 270_{10}$
- 3) 명령어 코드 11011_111_1011_1110 실행 후 PC는? $206 + (-66) = 140_{10}$

베이스 레지스터 주소지정방식 (스택용)



Base Register: 스택의 기준점 지정

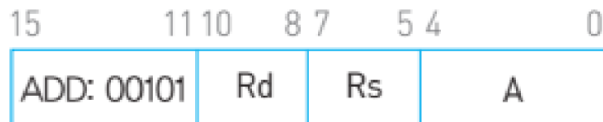
유효 주소 = (BR) + A (정수, 부호 확장 계산)

기준점의 위 아래 액세스 가능

× 베이스 레지스터 주소지정방식

[예제 6-7]

- 단어 크기와 명령어 크기가 16비트
- 기억장치 용량 64K단어
- 베이스 레지스터 주소지정방식 명령어: $Rd \leftarrow Rd + Mem((Rs)+A)$



(a) 명령어 형식

번호	데이터
...	...
3	521
2	518
1	124
0	21

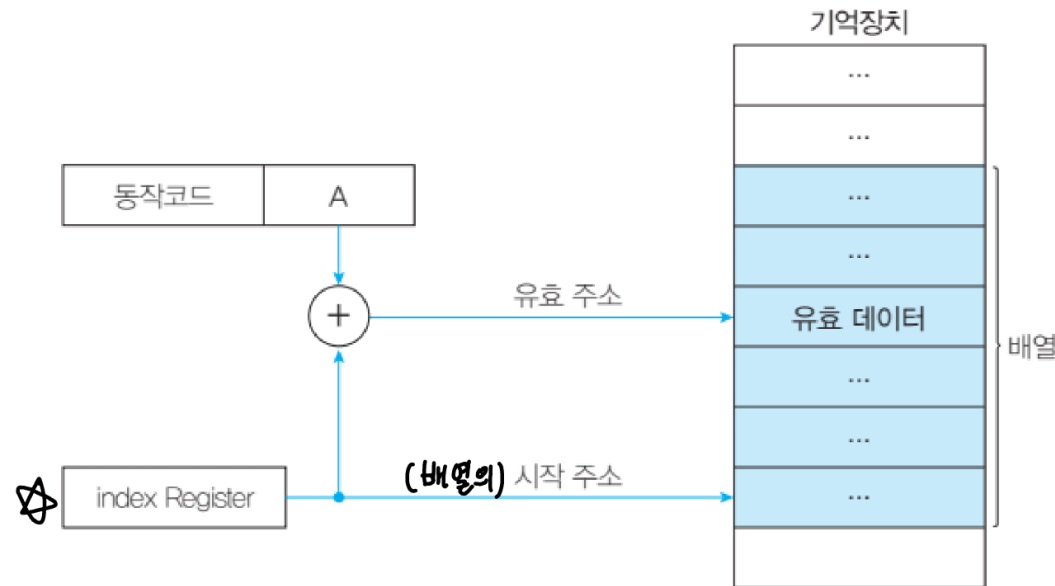
(b) 레지스터 상태

주소	데이터
...	...
521	22
520	33
519	44
518	55

(c) 기억장치 상태

- 1) 프로세서의 레지스터 수는?
- 2) 명령어 코드 00101_000_010_00001 실행 후 변경되는 레지스터 값은?
- 3) 명령어 코드 00101_001_011_11110 실행 후 변경되는 레지스터 값은?

인덱싱 (배열 다룰 때 쓰임)



유효 주소 = (Index Register) + A (부호 없는 수, 부호 확장 계산)

자동 인덱싱 (auto indexing)

- Pre-indexing: 인덱스 레지스터 갱신 후 기억장치 액세스) $i++$ 와 $++i$;
- Post-indexing: 기억장치 액세스 후 인덱스 레지스터 갱신) i 의 차이

X 인덱싱

[예제 6-8]

- 명령어 동작코드 11101: $Rd \leftarrow \text{Mem}((IX) + (Rs))$
- A 필드가 0이면, IX를 증가하지 않는다.
- A 필드가 1이면, 기억장치 액세스 후 IX를 증가한다.



(a) 명령어 형식

번호	데이터
...	...
3	500
2	13
1	12
0	11

(b) 레지스터 상태

주소	데이터
514	...
513	22
512	33
511	44
510	55

(c) 기억장치 상태

- 1) 명령어의 주소지정방식과 주소 필드 수는?
- 2) 명령어 코드 11101_0_000_011_001 실행 후 변경되는 레지스터 값은?
- 3) 명령어 코드 11101_1_000_011_010 실행 후 변경되는 레지스터 값은?

6.4 오퍼랜드 저장

오퍼랜드(동작의 대상)의 두 가지 의미

- 명령어 형식에서 동작의 대상을 표현하는 필드
- 동작 코드가 실제로 처리하는 유효 데이터

학습 목표

- 여러 바이트로 구성된 단어를 기억장치에 저장하는 방법을 이해한다.

내용

X 6.4.1 데이터 정렬

- 6.4.2 바이트 저장 순서

6.4.1 데이터 정렬

기억장치 구성

- 기억장치: 바이트 단위 주소 지정
- 단어(word)는 n 바이트에 분산
- 단어는 $\text{Mem}[A+0] \sim \text{Mem}[A+(n-1)]$ 번지에 연속적으로 저장되어 있음.

단어 크기에 정렬 (word-size aligned)

- n 의 배수로 시작하는 주소에 배정
- 단어가 배정된 시작 주소 A 는 n 의 배수 (단, $n=2^k$)

단어 액세스 방법

- 정렬된 n 바이트 단어를 한 번에 액세스할 수 있다.
- 정렬되지 않은 단어는 두 번에 걸쳐 읽은 후 조합해야 한다.

단어 크기에 정렬

4GB 용량, 4바이트 단어

A[31:2]	A ₁ A ₀ =11	A ₁ A ₀ =10	A ₁ A ₀ =01	A ₁ A ₀ =00	
0000_0000h	D[31:24]	D[23:16]	D[15:8]	D[7:0]	
0000_0004h	D[31:24]	D[23:16]	D[15:8]	D[7:0]	정렬된 단어
0000_0008h	D[31:24]	D[23:16]	D[15:8]	D[7:0]	
0000_000ch	D[31:24]	D[23:16]	D[15:8]	D[7:0]	
0000_0010h	D[31:24]	D[23:16]	D[15:8]	D[7:0]	정렬되지 않은 단어
0000_0014h	D[31:24]	D[23:16]	D[15:8]	D[7:0]	
0000_0018h	D[31:24]	D[23:16]	D[15:8]	D[7:0]	
0000_001ch	D[31:24]	D[23:16]	D[15:8]	D[7:0]	
...	D[31:0] ...				

[예제 6-9] 프로세서가 단어를 한 번에 액세스 할 수 있는가?

- 주소 0000_0004h, 0000_0005h, 0000_0006h, 0000_0007h 번지 단어
- 주소 1000_000eh, 1000_000fh, 1000_0010h, 1000_0011h 번지 단어
- 주소 1234_1e08h, 1234_1e09h, 1234_1e0ah, 1234_1e0bh 번지 단어
- 주소 b800_a003h, b800_a004h, b800_a005h, b800_a006h 번지 단어

6.4.2 바이트 순서

표준은 없음

Big: big한 쪽으로 가면서 읽기

Little: little한 쪽으로 가면서 읽기

바이트 순서 (byte order)

- 한 개의 단어가 한 바이트 이상의 기억장치 공간을 차지할 때,
- 단어를 구성하는 바이트를 주어진 공간에 배치하는 방법

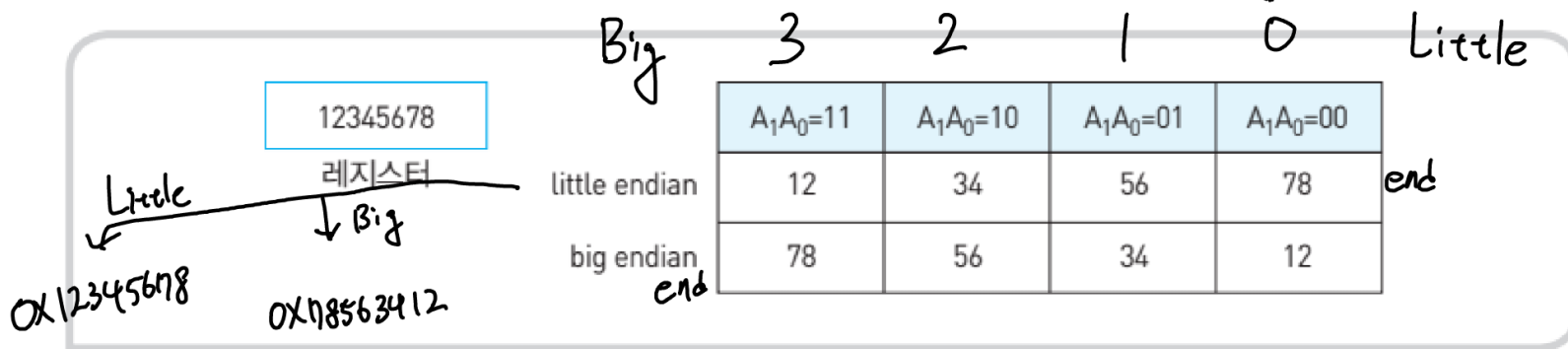


그림 6-21 바이트 순서

Little endian

- 주소 값이 작은 장소에 무게가 작은 수 저장
- 예) Intel X86, ARM

Big Endian: Little 반대

Little Endian: 주소 큰곳에 무게 큰 수
 ↳ 주소가 작은 쪽이 끝

Big endian

- 주소 값이 작은 장소에 무게가 큰 수 저장
- 예) Motorola 68XXX, MIPS, 인터넷

Big: ←로 읽기

Little: →로 읽기

RIP

RIP

바이트 저장 순서 예제

[예제 6-10]

- 1) 주소 1200h부터 1203h까지 데이터를 리틀 엔디언으로 해석하라.
- 2) 주소 1204h부터 1207h까지 데이터를 빅 엔디언으로 해석하라.

	주소	데이터		주소	데이터	
Big	1203h	55	Big	1207h	66	Big
	1202h	44		1206h	77	
	1201h	33		1205h	88	
Little	1200h	22	Little	1204h	99	Little

[풀이]

- 1) 주소 값이 작은 1200h에 무게가 작은 수: 55443322
- 2) 주소 값이 작은 1204h에 무게가 큰 수: 99887766

6.5 명령어 종류

명령어 설명서(instruction datasheet)

학습 목표

- 프로세서가 제공하는 명령어의 동작을 설명할 수 있다.

내용

- 6.5.1 데이터 전달 명령어
- 6.5.2 입출력 명령어
- 6.5.3 산술 연산 명령어
- 6.5.4 논리 연산 명령어
- 6.5.5 데이터 변환 명령어
- 6.5.6 상태 레지스터 조작 명령어
- 6.5.7 분기 명령어
- 6.5.8 서브루틴 호출 명령어
- 6.5.9 인터럽트 명령어
- 6.5.10 시스템 제어 명령어

6.5.1 데이터 전달 명령어

적재(load): $\text{Reg} \leftarrow \text{Mem}$

저장(store): $\text{Mem} \leftarrow \text{Reg}$

이동(move): $\{\text{Reg}, \text{Mem}\} \leftarrow \{\text{Reg}, \text{Mem}\}$

교환(exchange): $\{\text{Reg or Mem}\} \leftrightarrow \{\text{Reg or Mem}\}$

스택: PUSH REG, POP REG

6.5.2 입출력 명령어

입력(input): $\text{Reg} \leftarrow \text{I/O port}$

출력(output): $\text{I/O port} \leftarrow \text{Reg}$

일반적으로

- 직접주소지정방식 또는 레지스터 간접 주소지정방식을 사용한다.
- 복잡한 주소지정방식을 사용하지 않는다.

6.5.3 산술 명령어

단항 연산 명령어

- 절대값(absolute)
- 음수(negate)
- 증가(increment)
- 감소(decrement)
- 제곱근(square root)

이항 연산 명령어

- 더하기
- 빼기
- 곱하기
- 나누기

데이터 형식: 부호 없는 수, 정수, 실수, 이진화 십진 코드(BCD)

데이터 크기: 8, 16, 32, 64비트

6.5.4 논리 연산 명령어

논리 이항 연산 명령어

- AND, OR, XOR
- NAND, NOR, XNOR

논리 단항 연산 명령어

- NOT
- Clear bit
- Set bit
- Shift: 논리 시프트, 산술 시프트, 회전

6.5.5 데이터 변환 명령어

데이터 형식 변환 명령어

- BCD $\leftrightarrow$ Binary
- 비트 크기 변환: 8 비트 $\leftrightarrow$ 16 비트 $\leftrightarrow$ 32 비트
- 형식 변환: 정수 $\leftrightarrow$ 단정도 실수 $\leftrightarrow$ 배정도 실수

번역 (translation) 명령어

- 인덱싱 주소지정방식 활용
- $\text{Reg} \leftarrow \text{표 참조}$

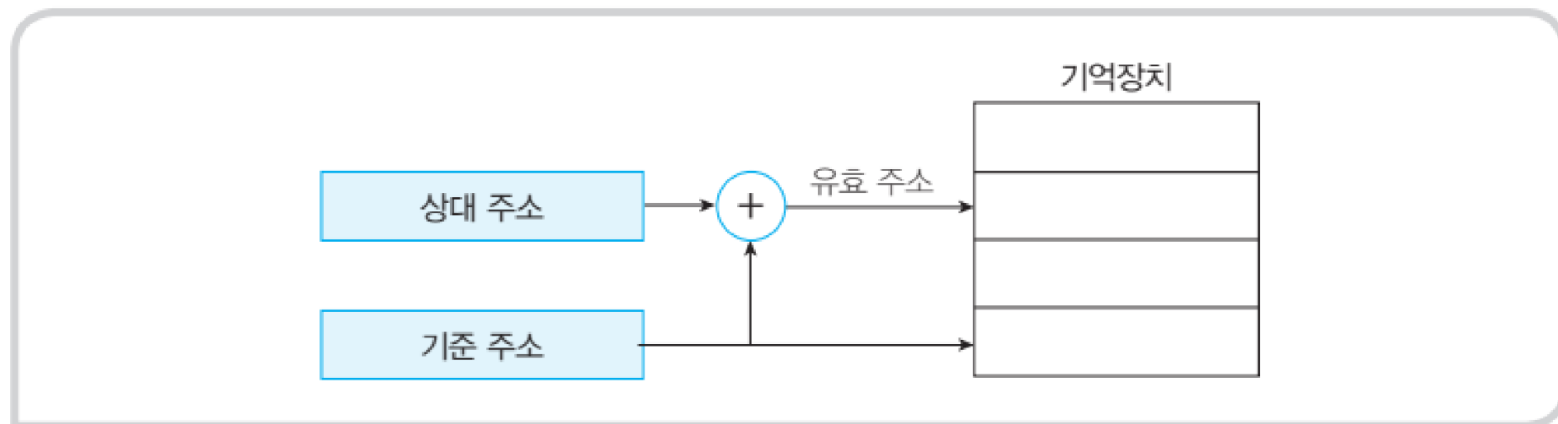


그림 6-23 번역 명령어

6.5.6 상태 레지스터 조작 명령어

Compare 명령어	compare A, B	// A – B
Test 명령어	test A, B	// A AND B

Set/reset carry flag 명령어

- stc (set carry flag)
- clc (clear carry flag)

Set/reset interrupt flag 명령어

- sti (set interrupt flag)
- cli (reset interrupt flag)

6.5.7 분기 명령어

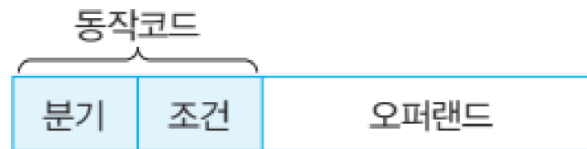


그림 6-24 분기 명령어 형식

오퍼랜드 = 분기 목적지 주소

- 직접 주소지정방식
- 레지스터 간접 주소지정방식
- 상대 주소지정방식

종류

- 무조건 분기 명령어: br / jmp
- 조건 분기 명령어
 - Branch taken : $PC \leftarrow$ 분기 목적지 주소
 - Branch not taken : $PC \leftarrow$ 다음 명령어 (변하지 않음)

조건 분기 예

[예제 6-11]

- 상태 레지스터 값이 다음과 같다.

$Z = 0, S = 0, C = 1, P = 0, OV = 1$

- 명령어 실행 후, PC 값을 구하라.
- 명령어 길이는 2바이트이고, 현재 실행하는 명령어는 1000번지에 저장되어 있고, 상대 주소지정방식을 사용한다.

- 1) `brz +100` // branch if $Z = 1$
- 2) `brnz +100` // branch if $Z = 0$
- 3) `brc +24` // branch if $C = 1$

[풀이]

- 1) branch (taken, not taken) ? PC = _____
- 2) branch (taken, not taken) ? PC = _____
- 3) branch (taken, not taken) ? PC = _____

조건 검사

표 6-1 상태 레지스터의 조건 검사

검사 방법	검사 조건	의미		니모닉 코드 예
플래그 검사	$Z = 1$	branch if zero flag equals 1		brz, jz
	$Z = 0$	branch if zero flag equals 0		brnz, jnz
	$S = 1$	branch if negative		brn, js
	$S = 0$	branch if positive		brp, jns
	$C = 1$	branch if carry		brc, jc
	$C = 0$	branch if no carry		brnc, jnc
	$P = 1$	branch if odd parity		brpo, jpo
	$P = 0$	branch if even parity		brpe, jpe
	$OV = 1$	branch if overflow		brov, jov
	$OV = 0$	branch if not overflow		brnow, jnov
부호 없는 수	$Z = 1$	equals	$A = B$	bre, je
	$(C = 1) \text{ and } (Z = 0)$	above	$A > B$	bra, ja
	$C = 1$	above or equal	$A \geq B$	brae, jae
	$(C = 0) \text{ or } (Z = 1)$	below or equal	$A \leq B$	brbe, jbe
	$C = 0$	below	$A < B$	brb, jb
부호 있는 수 (정수, 실수)	$Z = 1$	equals	$A = B$	bre, je
	$Z \text{ or } (S \oplus OV) = 0$	greater	$A > B$	brgt, jge
	$S \oplus OV = 0$	greater or equal	$A \geq B$	brge, jge
	$Z \text{ or } (S \oplus OV) = 1$	less or equal	$A \leq B$	brle, jle
	$S \oplus OV = 1$	less	$A < B$	brlt, jlt

조건 검사

[예제 6-12]

- <표 6-1>의 조건 검사 명령어를 모두 구현하려고 때,
- [그림 6-24]의 조건 필드에 최소 몇 비트를 할당해야 하는가?
- 무조건분기는 고려하지 않는다.

[풀이]

- 검사할 조건의 전체 수를 구하고 중복 조건을 빼서 조건의 수를 구한다.
- <표 6-1>의 조건 검사 수는 모두 20 개이다.
- 이 중에서 다음 명령어들은 조건이 같기 때문에 하나로 취급할 수 있다.
 - Z = 1: 플래그 검사 brz, 부호 없는 수 bre, 부호 있는 수 bre
 - C = 1: 플래그 검사 brc, 부호 없는 수 brae
 - C = 0: 플래그 검사 brnc, 부호 없는 수 brb
- 따라서 조건의 수 = $20 - 4 = 16$ 개이고,
- $2^4 = 16$ 이므로, 조건을 표현하는데 필요한 비트는 최소 4 비트.

6.5.8 서브루틴 호출 명령어

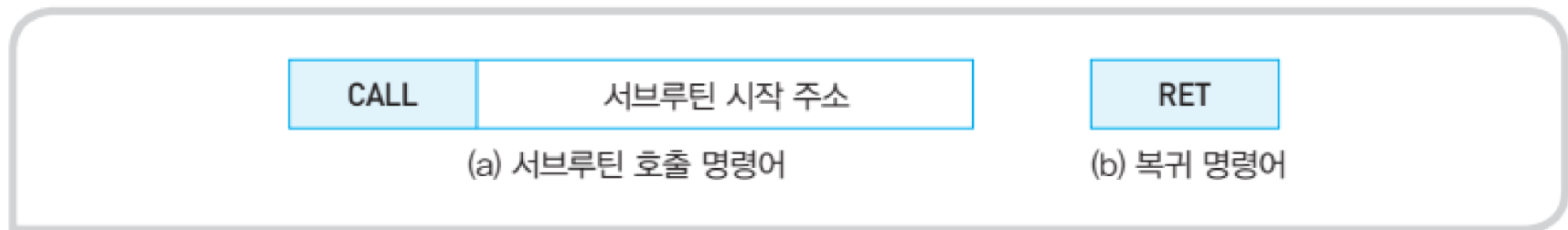


그림 6-25 서브루틴 호출과 복귀 명령어의 형식

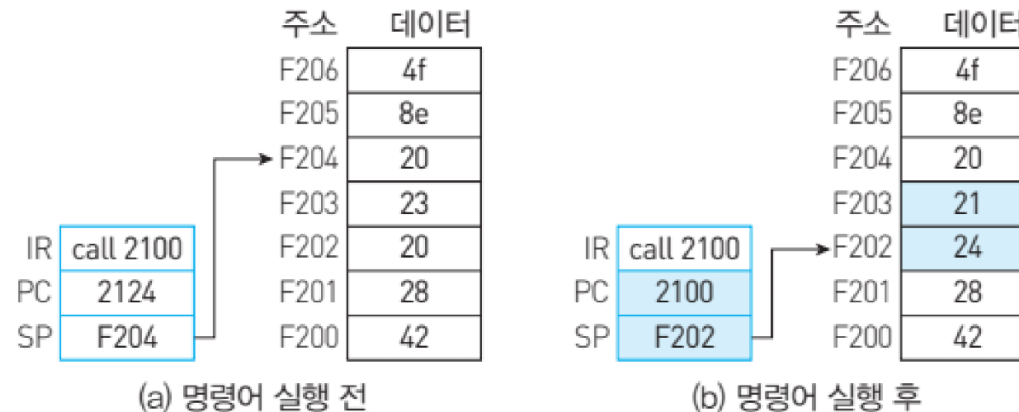
call [서브루틴 시작 주소]

```
PUSH PC    // 복귀 주소 저장  
PC ← 서브루틴 시작 주소
```

ret

```
POP PC     // PC 복구
```

서브루틴 호출 및 복귀 예제



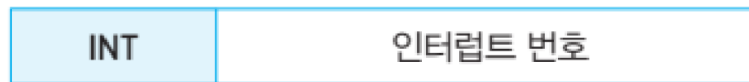
[예제 6-13]

- CALL 명령어의 동작코드는 6비트이고, 그 값은 001100_2 이다.
- CALL 명령어의 오퍼랜드 필드는 10비트 상대 주소지정방식으로 표현된다.
- 기억장치는 바이트 단위이고, 단어 크기와 명령어 길이는 2바이트이다.
- 기억장치는 리틀 엔디언 방식으로 데이터를 저장한다.
- 스택은 2바이트 크기로 정렬되어 있다.
- 푸시 동작은 $(SP \leftarrow SP - 2 / \text{Mem}[SP+1, SP] \leftarrow \text{오퍼랜드})$ 를 실행한다.

- 1) "CALL 2100" 명령어가 저장되어 있는 기억장치 주소를 구하라.
- 2) "CALL 2100" 명령어에 대한 명령어 코드를 16진수로 적어라.
- 3) 명령어 실행 후 상태를 그려라.

6.5.9 인터럽트 명령어

프로그래머가 인터럽트 호출 가능: 소프트웨어 interrupt
운영체제 기능 호출 위해 SW interrupt: 시스템 호출
System Call



(a) 인터럽트 요청 명령어



(b) 인터럽트 복귀 명령어

그림 6-27 인터럽트 요청과 복귀 명령어

소프트웨어 인터럽트

- System call

int n

```
PUSH PC    // 프로그램 카운터 저장
PUSH SR    // 상태 레지스터 저장
PC ← 인터럽트 서비스 루틴 시작 주소
```

인터럽트 복귀 명령어

iret

```
POP SR     // 상태 레지스터 복구
POP PC     // PC 복구
```

6.5.10 시스템 제어 명령어

시스템 관리, 운영체제 기능 지원 명령어

- Halt 시스템 정지 프로세서 정지, 다시 수행하려면 리셋
- Wait 이벤트 대기 이벤트 발생 시까지 수행X, 주로 개발과정에서 테스트용
- Nop 클럭 소모 → No Operation ; 프로세서가 잉여한 시간을 보내야 할 때 사용
1클럭동안 멍때림

프로세서 동작 모드(operation mode)

- 시스템 모드(system mode, 커널 모드, 운영체제 모드)
- 사용자 모드(user mode)

특권 명령어(privileged instruction)

- 시스템 모드에서만 실행할 수 있는 명령어
- 특수 레지스터 관리, 기억장치 관리, 입출력 명령어 등

6.6 요약

6.1 명령어 특성

- 동작 코드 + 오퍼랜드

6.2 주소의 수

- 3-주소, 2-주소, 1-주소 0-주소명령어 형식

6.3 주소지정방식

- 즉치, 직접, 간접, 레지스터, 레지스터 간접,
- 상대, 베이스 레지스터, 인덱싱

6.4 오퍼랜드 저장

- 데이터 정렬(word-size aligned), 바이트 저장 순서

6.5 명령어 종류

- 데이터 전달, 입출력,
- 산술 연산, 논리 연산, 데이터 변환, 상태 레지스터 조작,
- 분기 및 조건분기, 서브루틴 호출 및 복귀, 인터럽트
- 시스템 제어 명령어

